

Plano de Entregas: Monetizacao

Integracao de pagamentos via Kiwify, controle de planos e paywall no bot do Telegram.

Escopo dividido em 3 etapas independentes, cada uma testavel e implantavel isoladamente.

Revisao 2 - corrige quatro pontos do plano inicial que nao funcionariam na arquitetura atual.

1. Resumo executivo

O LekoAI Finance hoje e aberto: qualquer pessoa que encontre o bot usa todos os recursos sem limite. Este plano introduz tres planos pagos e um plano gratuito com cota, mantendo o bot funcionando normalmente durante toda a transicao.

A venda acontece fora do bot, numa landing page da Kiwify. Quando o pagamento e aprovado, a Kiwify avisa o sistema automaticamente e o comprador recebe um link que ativa o plano dentro do Telegram.

Etapas	Entrega	Bloqueia usuario?	Risco
0	Fechar a policy aberta em chamados (fora do escopo, mas exposto hoje)	Nao	Baixo
1	Fundacao e modo sombra: estrutura de planos, leitura, comandos de diagnostico	Nao	Baixo
2	Paywall ativo: limites passam a valer, mensagens de venda com link de checkout	Sim	Medio
3	Webhook Kiwify e ativacao automatica do plano apos a compra	Nao (libera)	Medio

A ordem importa. A Etapa 0 nao faz parte da monetizacao, mas trata um problema de seguranca que a auditoria encontrou aberto em producao (secao 4.4) e por isso vem primeiro. A Etapa 1 sobe sem bloquear ninguem e serve para medir o uso real antes de escolher os limites definitivos. A Etapa 3 pode ser feita antes da 2 se houver pressa para comecar a vender, mas nao o contrario.

2. Correcoes em relacao ao plano inicial

A auditoria do codigo encontrou quatro pontos do plano v1 que nao funcionariam. Registrados aqui para nao serem reintroduzidos:

1. Criar `api/payment_webhook.py` daria 404. A Vercel neste projeto roda com entypoint unico (`pyproject.toml`, `[tool.vercel]` `entrypoint = api.telegram:handler`). Toda rota nova precisa ser despachada dentro de `api/telegram.py`. Ja houve dois commits brigando com isso (4f43741, ab505d9).
2. Nao havia ponte entre Kiwify e Telegram. A Kiwify entrega e-mail e CPF, nunca o `telegram_id`. Resolvido com token de ativacao e deep link.
3. O schema v1 assumia degustacao por tempo (`NOW() + 7 days`), mas a degustacao escolhida e por quantidade. Com o SQL antigo todo usuario seria bloqueado no oitavo dia sem ter usado nada.
4. O token de ativacao nao pode morar na tabela `users`: o aviso de pagamento chega antes de o comprador existir na base, porque ele pode nunca ter falado com o bot.

3. Principio de arquitetura: uma porta so

Existem dois caminhos ate o relatorio, e apenas um passa por `process_message`. O clique nos botoes de formato (`fmt_pdf`) e tratado direto em `api/telegram.py` e em `tools/telegram_bot.py`, sem passar pelo roteador.

Consequencia pratica

Se o controle de acesso ficar apenas no `process_message`, um usuario do plano gratuito clica num botao antigo do historico e recebe o relatorio de seis meses. O controle precisa ficar dentro de `gerar_relatorio_por_formato`, que e o ponto comum aos dois caminhos.

Regra do plano: toda decisao de acesso vive em `tools/subscription.py`, na funcao `check_access()`. Nenhum outro arquivo decide se o usuario pode. Isso tambem e o que torna o modo de teste trivial: desligar o `paywall` e um `if` no topo de uma funcao.

4. Resultado da auditoria de segurança (27/08/2026)

A auditoria que este plano exigia como pre-requisito foi executada (relatorio completo em Verificacoes.txt na raiz do projeto). Ela liberou um bloqueio, corrigiu uma premissa errada do plano e encontrou um problema real fora do escopo da monetizacao.

4.1 RESOLVIDO - o paywall nao e contornavel pelo navegador

A preocupacao original era que o papel anonimo pudesse alterar a tabela users e o proprio usuario se promovesse a vitalicio pelo navegador. Nao acontece: users e gastos tem RLS ligado e NENHUMA policy, o que no Postgres significa negar tudo. Verificado na pratica: o papel anonimo enxerga zero linhas nas duas tabelas.

Etapa 2 desbloqueada

O que era pre-requisito bloqueante deixou de ser. A Etapa 2 pode seguir sem esperar por mudancas de permissao em users.

4.2 ATENCAO - os GRANTS do papel anonimo estao abertos

Padrao de fabrica do Supabase: anon e authenticated tem SELECT, INSERT, UPDATE e DELETE em todas as tabelas do schema public. Hoje isso e inofensivo porque o RLS nega, mas significa que o RLS e a UNICA trava. Uma policy permissiva criada sem cuidado no futuro abre tudo de uma vez, porque o GRANT por baixo ja permite.

Por isso as migrations das Etapas 1 e 3 passam a revogar os GRANTS explicitamente, em vez de confiar apenas na ausencia de policy. Defesa em profundidade: passariam a ser necessarios dois erros para vazar, nao um.

4.3 CORRECAO DE PREMISSE - SUPABASE_KEY ja e service_role

O plano dizia para usar SUPABASE_SERVICE_KEY em vez da SUPABASE_KEY atual. A premissa estava errada: verificado que a SUPABASE_KEY tem prefixo sb_secret_, ou seja ja e o equivalente da service_role e ja ignora o RLS. E exatamente por isso que db_manager.py consegue escrever em tabelas que negam tudo.

Nao existe chave nova a introduzir. O que resta e um problema de nome: 'Chave de API do Supabase' (README.md:79) e generico o bastante para alguem colar uma chave SECRETA num arquivo de frontend achando que e a publica. O rename e desejavel, mas se o codigo mudar e a variavel nao for criada na Vercel, o bot cai em producao. Fazer em duas fases:

```
# fase 1: aceita os dois nomes, com fail-fast
key = os.environ.get("SUPABASE_SERVICE_KEY") or os.environ.get("SUPABASE_KEY")
if not key:
    raise RuntimeError("SUPABASE_SERVICE_KEY/SUPABASE_KEY ausente")

# fase 2: criar a variavel nova na Vercel, verificar, e so entao remover o nome antigo
```

Isso casa com o item #2 do roadmap de melhorias (fail-fast quando SUPABASE_URL ou a chave sao nulas). Vale fazer junto, num commit proprio.

4.4 CRITICO - policy aberta em chamados (fora do escopo, mas vazando hoje)

```
policy : "Allow all for service role" on public.chamados
roles  : {public}    <- em Postgres significa TODO MUNDO, nao o papel de servico
cmd    : ALL         qual: true      with_check: true
```

O nome da policy diz 'service role', mas ela foi criada sem a clausula TO service_role. Em Postgres, {public} quer dizer todo mundo. O nome enganou; o efeito e irrestrito.

Combinado com public/dashboard.html, que fala direto com o Supabase pelo navegador nas linhas 221 (update) e 229 (delete): qualquer pessoa que abra a rota /dashboard e leia o codigo-fonte do HTML consegue alterar ou apagar todos os chamados. A chave publicavel nao e o problema, ela foi feita para ficar exposta; quem deveria proteger e a policy.

Tratado como Etapa 0

Nao e monetizacao, mas e pequeno e esta aberto agora. Vai antes de tudo, com ordem obrigatoria: derrubar a policy antes de migrar o dashboard quebra o dashboard.

4.5 REGRA NOVA - nunca falhar aberto

A auditoria encontrou este padrao em tools/chamados_service.py, linhas 35 a 37:

```
if not DASHBOARD_TOKEN:
    return True          # <- libera escrita se a variavel estiver vazia
```

Verificado que DASHBOARD_TOKEN nao existe no .env local, entao localmente a API aceita escrita de qualquer origem. Na Vercel precisa ser conferido.

Regra para este plano: validacao ausente significa NEGAR

O payment_service.py nao pode repetir esse padrao. Sem KIWIFY_WEBHOOK_TOKEN configurado, o webhook rejeita tudo. Um webhook de pagamento que falha aberto significa que qualquer pessoa manda um aviso de compra falso e ganha plano vitalicio de graca.

4.6 AINDA ABERTO - confirmacoes no painel da Kiwify

O que foi documentado ate agora foi ONDE verificar no painel, nao os valores encontrados. Estes continuam desconhecidos, e a Etapa 3 nao pode ser codificada sem eles:

- ☐ Algoritmo e formato exatos da assinatura: HMAC-SHA1 ou SHA256? hex ou base64? sobre o corpo bruto?
- ☐ Nome do parametro que carrega a assinatura (signature na URL?)
- ☐ Strings exatas dos eventos: aprovado, renovacao, estorno, chargeback, cancelamento
- ☐ Identificadores de produto e oferta dos tres planos, para mapear o tipo de plano

Como fechar isso em cerca de 10 minutos

Criar uma URL em webhook.site, cadastrar como webhook na Kiwify e disparar um evento de teste de cada tipo. Copiar o corpo e a query string recebidos entrega todos os itens acima de uma vez, sem depender de a documentacao estar atualizada.

4.7 Variaveis de ambiente

```
PAYWALL_ENABLED=true           # kill switch global do paypal
ADMIN_TELEGRAM_IDS=            # ids que nunca sao bloqueados (separados por virgula)
CHECKOUT_URL=                  # landing page da Kiwify
KIWIFY_WEBHOOK_TOKEN=          # sem este valor o webhook NEGA tudo (ver 4.5)
DASHBOARD_TOKEN=               # conferir se existe na Vercel (ver 4.5)
```

Nao ha SUPABASE_SERVICE_KEY a criar: a SUPABASE_KEY atual ja e service_role (ver 4.3). Na Vercel, PAYWALL_ENABLED deve ser escopado por ambiente: false em Preview, true em Production. As demais valem para os dois.

5. Etapas de entrega

0

Fechar a policy aberta em chamados

Fora do escopo de monetizacao. Vai primeiro porque esta vazando agora.

A ORDEM IMPORTA: derrubar a policy antes do passo 1 quebra o dashboard.

1. Apontar public/dashboard.html para /api/chamados em vez de criar o cliente Supabase no navegador. O backend seguro JA EXISTE (tools/chamados_service.py mais o dispatcher em api/telegram.py), portanto o acesso direto do HTML e redundante. Trocar as chamadas db.from('chamados') por fetch. Isso remove a chave do HTML e elimina a escrita anonima.
2. Confirmar DASHBOARD_TOKEN na Vercel e inverter o default de auth_ok() para negar quando a variavel estiver ausente.
3. So depois de 1 e 2 testados, remover a policy aberta: drop policy "Allow all for service role" on public.chamados. Se ainda quiser leitura publica, manter apenas a policy de leitura e restringi-la a TO anon FOR SELECT.
4. Opcional, no mesmo lote: rename de SUPABASE_KEY para SUPABASE_SERVICE_KEY em duas fases (ver 4.3), somado ao fail-fast do item #2 do roadmap.

Critério de pronto

- ☐ Dashboard funcionando sem nenhuma chave do Supabase no HTML
- ☐ PATCH e DELETE em /api/chamados sem token devolvendo 401
- ☐ Papel anonimo sem conseguir escrever em chamados

1

Fundacao e modo sombra

Sobe em producao sem bloquear ninguem. Mede antes de decidir.

Objetivo

Criar toda a estrutura de planos e a funcao de decisao, mas com o paywall desligado. O check_access roda e registra em log o que teria bloqueado, sem bloquear. Isso permite ver numeros reais de uso antes de fixar o limite de 20 lancamentos.

Banco de dados

```

ALTER TABLE users
ADD COLUMN IF NOT EXISTS plan_type VARCHAR(20) NOT NULL DEFAULT 'free',
    -- 'free' | 'plus_monthly' | 'plus_annual' | 'lifetime'
ADD COLUMN IF NOT EXISTS subscription_expires_at TIMESTAMPTZ,
    -- NULL em 'free' e em 'lifetime' (vitalicio nao expira)
ADD COLUMN IF NOT EXISTS customer_email VARCHAR(255);

CREATE INDEX IF NOT EXISTS idx_gastos_user_created ON gastos(user_id, created_at);

-- Defesa em profundidade (ver 4.2): hoje so o RLS trava e os GRANTS de
-- fabrica deixam anon com UPDATE. Revogar explicitamente as colunas de plano.
REVOKE UPDATE (plan_type, subscription_expires_at, customer_email)
    ON TABLE public.users FROM anon, authenticated;

```

Alteracoes no codigo

Arquivo	O que muda
tools/subscription.py (novo)	check_access(user_row, acao) -> Verdict. Modulo puro, sem HTTP. Ordem: kill switch, bypass de admin, vitalicio, assinatura valida, cota do plano gratuito.
tools/db_manager.py	get_or_create_user_row() devolvendo a linha inteira; get_or_create_user() vira wrapper de uma linha para nao quebrar chamadores; contar_lancamentos_mes(); aplicar_plano(); revogar_plano().
tools/message_handler.py	Passa a usar get_or_create_user_row. Novos comandos /plano e /dev. Log de sombra nos pontos que futuramente vao bloquear.

Cuidado com o fuso horario

contar_lancamentos_mes precisa usar America/Sao_Paulo para calcular a virada do mes. O projeto ja tem tzdata e ja corrigiu um bug de fuso no commit b21714c. Um lancamento feito dia 1 as 00h30 no horario de Brasilia precisa contar no mes novo, nao no anterior.

Decisao de desenho: a contagem e derivada da tabela gastos em vez de um contador em users. Nao precisa resetar na virada do mes e nunca dessincroniza. Efeito colateral aceito: apagar um lancamento devolve cota.

Como testar

- Testes automatizados de check_access: matriz de estados de plano por acao (funcao pura, facil de cobrir).
- Teste de regressao de fuso: lancamento em 01/mm as 00h30 BRT conta no mes correto.
- Manual: /plano mostra plano, validade e contagem corretos para varios usuarios.
- Manual: nenhum usuario e bloqueado; os logs de sombra aparecem.

Criterio de pronto

- ☐ Migracao aplicada e auditoria de RLS da secao 4.1 concluida
- ☐ /plano responde corretamente para plano gratuito, pago e vitalicio
- ☐ Logs de sombra registrando o que teria sido bloqueado

☐ Zero usuarios bloqueados em producao

2

Paywall ativo

Os limites passam a valer. Etapa de maior risco de percepção.

Objetivo

Ligar `PAYWALL_ENABLED` e fazer os limites valerem: 20 lançamentos por mês e relatório de no máximo 7 dias no plano gratuito. Ao bater no limite, o bot responde com uma mensagem de venda e um botão levando para a landing page.

Os três pontos de controle

1. Ramo registrar do `process_message`: bloqueia o lançamento 21 do mês.
2. Ramo relatório do `process_message`: bloqueia período acima de 7 dias.
3. Dentro de `gerar_relatorio_por_formato`: fecha o furo dos botões de formato, que não passam pelo `process_message` nem no webhook nem no bot local.

Não unificar as duas constantes de 7 dias

`LIMITE_DIAS_PDF = 7` (já existe) significa 'relatório vira arquivo em vez de texto'.
`LIMITE_DIAS_RELATORIO_FREE = 7` (novo) significa 'precisa pagar'. Mesmo número, significados diferentes. Manter os dois nomes separados, senão mudar a regra comercial muda o formato do relatório sem querer.

Mensagens de bloqueio

O Verdict devolvido por `check_access` carrega o motivo (`cota_estourada`, `periodo_longo`, `plano_expirado`) para o paywall montar a mensagem certa. Botão de checkout via `inline_keyboard` com url, reaproveitando o padrão de `send_message_with_buttons`.

Como testar

Com o comando `/dev`, simular cada estado e conferir a resposta:

Estado simulado	Resultado esperado
Gratuito, 19 de 20 usados	Lançamento passa; /plano mostra 20 de 20 depois
Gratuito, 20 de 20 usados	Lançamento bloqueado com botão de checkout
Gratuito, relatório de 7 dias	Relatório entregue normalmente
Gratuito, relatório de 8 dias	Bloqueado com mensagem de venda
Gratuito clicando em botão antigo	Bloqueado (este é o furo da seção 3)
Plus expirado ontem	Bloqueado como plano expirado
Vitalício	Nunca bloqueado, sem data de validade
Id em <code>ADMIN_TELEGRAM_IDS</code>	Nunca bloqueado, em qualquer ambiente
<code>PAYWALL_ENABLED=false</code>	Tudo liberado, independente do plano

Criterio de pronto

- ☐ Todos os nove casos da tabela acima verificados manualmente no bot de dev
- ☐ Etapa 0 concluida (a chave do Supabase nao esta mais no HTML)
- ☐ Testado em Preview antes do merge na main

Nota: a auditoria de RLS que antes bloqueava esta etapa ja foi feita e nao encontrou problema em users. Ver secao 4.1.

Objetivo

Receber o aviso de pagamento da Kiwify, registrar a compra e permitir que o comprador ative o plano no Telegram por meio de um link com token.

Por que uma tabela separada

O aviso da Kiwify chega antes de o comprador existir na tabela users, porque ele pode nunca ter falado com o bot. Nao ha linha para atualizar. Por isso o registro vai para uma tabela propria, e o vinculo com users acontece depois, na ativacao.

```
CREATE TABLE IF NOT EXISTS pagamentos (  
  id                BIGSERIAL PRIMARY KEY,  
  order_id          TEXT NOT NULL UNIQUE,    -- idempotencia  
  customer_email    TEXT NOT NULL,  
  plan_type         TEXT NOT NULL,  
  status            TEXT NOT NULL,          -- 'pago' | 'estornado'  
  activation_token  TEXT UNIQUE,  
  user_id           BIGINT REFERENCES users(id), -- NULL ate a ativacao  
  event_raw         JSONB,  
  created_at        TIMESTAMPTZ DEFAULT NOW(),  
  activated_at      TIMESTAMPTZ  
);  
  
CREATE INDEX IF NOT EXISTS idx_pagamentos_token ON pagamentos(activation_token);  
  
-- Tabela nova tambem nasce com os GRANTS de anon (ver 4.2): revogar faz  
-- parte da migration. Nenhuma policy: so a chave secreta acessa.  
ALTER TABLE public.pagamentos ENABLE ROW LEVEL SECURITY;  
REVOKE ALL ON TABLE public.pagamentos FROM anon, authenticated;
```

order_id UNIQUE resolve idempotencia de graca: se a Kiwify reenviar o mesmo evento, o insert conflita e e ignorado, em vez de somar doze meses duas vezes num plano anual.

Fluxo de ativacao

1. O comprador paga na landing page da Kiwify.
2. A Kiwify envia o aviso para /api/pagamento com a assinatura de seguranca.
3. O sistema valida a assinatura sobre o corpo bruto da requisicao, antes de interpretar o JSON.
4. Registra a compra e gera um token de ativacao.
5. Responde 200 rapidamente, para a Kiwify nao reenviar.
6. A pagina de obrigado mostra um link do tipo <https://t.me/SeuBot?start=TOKEN>.
7. O comprador clica, o bot recebe /start TOKEN, casa o token, grava o plano e queima o token.

Por que token e não e-mail

Se o bot apenas perguntasse o e-mail do comprador, qualquer pessoa que descobrisse o e-mail de um cliente ativaria o plano no lugar dele. O token é de uso único e só quem concluiu a compra recebe.

Alterações no código

Arquivo	O que muda
tools/payment_service.py (novo)	Espelha o contrato de tools/chamados_service.py: recebe headers e corpo, devolve (status, dict). Valida HMAC, mapeia produto para plano, calcula validade, trata estorno, chargeback e cancelamento.
api/telegram.py	Novo _is_pagamento() despachando /api/pagamento. Precisa ficar junto do _is_chamados(), ou seja ANTES da checagem do WEBHOOK_SECRET: se cair depois, a Kiwify recebe 401 porque não envia o header do Telegram.
tools/message_handler.py	/start passa a aceitar payload. Hoje faz startswith('/start') e descarta o resto.
scripts/fake_kiwify.py (novo)	Simula a compra calculando a assinatura HMAC real, para o teste passar pelo mesmo caminho de validação do código de produção.

Sem modo sem validação de assinatura

O script de teste calcula a assinatura de verdade em vez de existir um atalho que pula a validação. Esse tipo de atalho vaza para produção e vira brecha de segurança.

Falhar fechado, ao contrário do auth_ok() atual

Sem KIWIFY_WEBHOOK_TOKEN configurado, o webhook rejeita tudo. É o oposto do padrão encontrado em chamados_service.py (ver 4.5), e a diferença importa: um webhook de pagamento que libera quando a validação está ausente entrega plano vitalício a quem enviar um aviso de compra falso.

Sobre tempo de resposta: o maxDuration de 10 segundos em vercel.json é suficiente, mas a Kiwify reenvia o aviso em qualquer resposta que não seja 2xx. Gravar no banco, responder 200 e só depois enviar a mensagem no Telegram. Relevante porque o timeout de 504 do commit 5e99e13 acabou de ser corrigido.

Critério de pronto

- ☐ scripts/fake_kiwify.py ativa um plano de ponta a ponta
- ☐ Assinatura inválida devolve 401
- ☐ Evento reenviado com o mesmo order_id não duplica a validade
- ☐ Estorno e chargeback revogam o acesso
- ☐ Uma compra real de R\$ 14,90 feita por você funciona

6. Ambiente de teste sem regras de pagamento

Requisito explicito: precisa existir um jeito de usar a aplicacao sem as regras de pagamento. Isso fica barato porque tools/telegram_bot.py e api/telegram.py compartilham a mesma process_message: um if num lugar so desliga tudo.

Camada	Mecanismo	Onde vale
1. Bypass de admin	ADMIN_TELEGRAM_IDS com o seu id do Telegram	Todos os ambientes, inclusive producao
2. Kill switch	PAYWALL_ENABLED=false	Local sempre; na Vercel apenas em Preview
3. Bot separado	Segundo token do BotFather no .env local	Somente desenvolvimento local

O segundo bot e requisito, nao conveniencia

Com um webhook registrado, o getUpdates do polling falha com 409 Conflict. Rodar tools/telegram_bot.py com o token de producao exigiria apagar o webhook, derrubando o bot de producao. Pedir um segundo bot ao BotFather e gratis e resolve.

Isolamento de dados

Decisao tomada: mesmo projeto Supabase para teste e producao. O bot de desenvolvimento tem outro telegram_id, entao vira outra linha em users e os dados de teste ficam naturalmente separados dos reais, sem precisar manter dois schemas em sincronia.

Comando /dev (restrito a admins)

```
/dev plano free          -> vira plano gratuito
/dev plano plus_expirado -> plus com validade em ontem
/dev plano lifetime       -> vitalicio
/dev lancamentos 19       -> simula 19 de 20 usados
/dev status               -> mostra o veredito do check_access
```

A checagem de admin e a primeira linha do handler. Assim o comando pode existir em producao sem risco, o que e o que permite testar pelo celular.

7. Riscos e mitigacoes

Risco	Impacto	Mitigacao
Usuario muda o proprio plano pelo navegador	Alto: paywall inutil	RESOLVIDO na auditoria: users nega tudo sob RLS. Mais os REVOKE das Etapas 1 e 3 como segunda camada
Policy aberta em chamados permite qualquer pessoa apagar dados	Alto: perda de dados, ja exposto	Etapa 0, antes da monetizacao, na ordem obrigatoria da secao 4.4

Risco	Impacto	Mitigacao
Validacao ausente libera acesso (falha aberta)	Alto: compra falsa vira vitalicio	Regra 4.5: payment_service nega sem token. Inverter tambem o auth_ok() atual
Policy permissiva futura abre tudo, porque os GRANTS estao abertos	Alto: vazamento total	REVOKE explicito nas migrations, para exigir dois erros e nao um
Rename da chave do Supabase derruba producao	Alto: bot fora do ar	Rename em duas fases lendo os dois nomes, com fail-fast (secao 4.3)
Furo do botao de formato antigo	Medio: relatorio longo de graca	Controle dentro de gerar_relatorio_por_formato, coberto por teste manual explicito
Rota de pagamento cai depois da checagem do WEBHOOK_SECRET	Alto: Kiwify recebe 401 e a compra nunca ativa	Despachar junto do _is_chamados(); teste com fake_kiwify antes do merge
Reenvio de evento da Kiwify duplica validade	Medio: cliente ganha meses a mais	order_id UNIQUE na tabela pagamentos
Formato da assinatura da Kiwify diferente do presumido	Alto: nenhuma compra ativa	Confirmar no painel antes de codificar; compra real de teste no criterio de pronto
Limite de 20 lancamentos errado para o uso real	Medio: churn ou receita perdida	Modo sombra na Etapa 1 mede antes de ligar; mudar o limite e trocar uma constante
Bloqueio de usuarios que ja usavam de graca	Medio: reacao negativa	Decidir politica de transicao antes da Etapa 2 (ver secao 8)

8. Pontos ainda em aberto

Tecnicos (bloqueiam a Etapa 3)

- Confirmacoes no painel da Kiwify: assinatura, eventos e IDs de produto. Ver secao 4.6, incluindo o caminho de webhook.site para fechar tudo de uma vez.
- Confirmar se DASHBOARD_TOKEN esta configurada na Vercel.

Comerciais (precisam de decisao antes da Etapa 2)

- Usuarios que ja usam o bot hoje sem limite: entram como gratuitos, ganham um periodo de cortesia ou recebem vitalicio como early adopters?
- A cota de 20 lancamentos reinicia no dia 1 de cada mes ou 30 dias apos o cadastro?
- O plano gratuito deve receber as dicas financeiras geradas por IA, ou isso e exclusivo dos pagos?
- Havera reembolso proporcional em caso de cancelamento do plano anual?

9. Politica de deploy

A branch main faz deploy automatico de producao. Para todas as etapas: subir primeiro em Preview, apontar o bot de desenvolvimento para a URL de preview, validar o criterio de pronto e so depois abrir o merge para a main.